DAY 78 | SORTING SUPPORT | WHITELIST SECURITY | DETERMINISTIC ORDERING

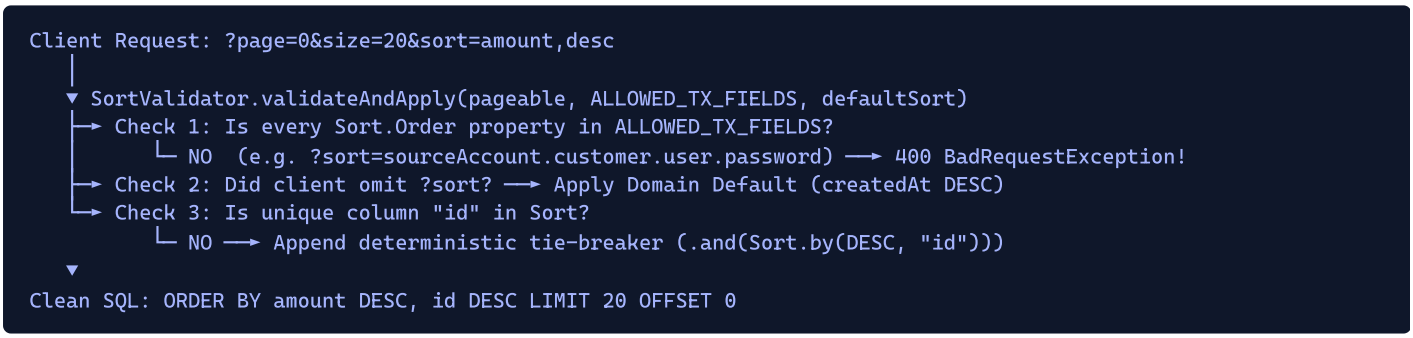
Dynamic Sorting, Allowlists & Tie-Breaker Stability

Study Notes & Planned Implementation Blueprint: Multi-Column Spring Data Sort Support, Preventing Arbitrary Property Traversal Attacks, Removing Rigid OrderBy Repository Names, and Aligning with Day 70 B-Tree Indexes.

PROJECT	CORE UTILITY	TIE-BREAKER	SAFETY NET
Enterprise Digital Banking	**SortValidator Helper**	**Secondary `id` Sort**	**PropertyReferenceEx**

1. Core Concepts: Why "Free" Spring Data Sorting Has Hidden Production Bugs

Once controllers accept `Pageable` (Day 77), Spring Web automatically binds query parameters like `?sort=amount,desc&sort=createdAt,asc` into a `Sort` object. However, passing raw client `Sort` objects straight into JPA repositories introduces **four classic enterprise hazards**:



Production Hazard	What Happens Without Day 78 Guardrails	Day 78 Architectural Solution
1. Relation Traversal / Info Leak	`?sort=customer.user.password` causes Hibernate to auto-`JOIN` `customers` and `users` and leak sort ordering of sensitive columns.	Strict per-entity `Set<String> ALLOWED_SORT_FIELDS` whitelist checked inside `SortValidator` before any SQL runs.
2. 500 on Unknown Property	`?sort=invalidCol,asc` throws `PropertyReferenceException`, hitting our 500 catch-all in `GlobalExceptionHandler`.	Intercept in `SortValidator` + add global handler for `org.springframework.data.core.PropertyReferenceException` (`400`).
3. Rigid Repository `OrderBy`	Our existing repo methods end in `...OrderByCreatedAtDesc`! Spring prepends that hardcoded column BEFORE dynamic `Pageable` sorts, breaking `?sort=amount,asc`!	Remove rigid `OrderBy...` suffixes from repository method names; pass dynamic `Sort` inside `Pageable` via `@PageableDefault` & `SortValidator`.
4. Shifting Rows Across Pages	Sorting solely by `createdAt` when 2 rows share the exact timestamp yields unstable MySQL row order across page 0 → page 1.	Automatically append deterministic primary-key tie-breaker (`id`) when `id` is not already in the sort orders.

2. Domain-by-Domain Sort Allowlists (Mapped to Real Entity Fields)

Every property below is verified directly against our JPA entities and aligned with our Day 70 B-Tree indexes:

Domain Entity	Allowed Sort Properties	Default Sort & Day 70 Index Alignment
`Transaction`	`id`, `createdAt`, `amount`, `transactionType`, `transactionStatus`, `transactionReference`	`createdAt DESC, id DESC` *(Uses `idx_tx_source_created`)*
`Account`	`id`, `accountNumber`, `balance`, `accountType`, `accountStatus`, `branch`, `createdAt`	`createdAt DESC, id DESC` *(Uses `idx_accounts_customer_id`)*
`Customer`	`id`, `firstName`, `lastName`, `city`, `state`, `kycStatus`, `createdAt`	`createdAt DESC, id DESC`
`LedgerEntry`	`id`, `createdAt`, `amount`, `entryType`, `balanceAfterEntry`	`createdAt DESC, id DESC` *(Uses `idx_ledger_account_created`)*
`AuditLog`	`id`, `createdAt`, `action`, `entityName`	`createdAt DESC, id DESC` *(Uses `idx_audit_entity_search`)*
`Loan` & `Emi`	**Loan:** `id`, `principalAmount`, `interestRate`, `tenureMonths`, `status`, `createdAt` **EMI:** `id`, `installmentNumber`, `dueDate`, `amount`, `status`	**Loan:** `createdAt DESC` **EMI:** `installmentNumber ASC, id ASC`
`Beneficiary`	`id`, `nickName`, `status`, `createdAt`	`createdAt DESC, id DESC`

3. Planned Code: Centralized `SortValidator` & Global Exception Guard

src/main/java/ ... /common/util/SortValidator.java NEW

```java
package org.example.enterprisedigitalbankingsystem.common.util;

import org.example.enterprisedigitalbankingsystem.exception.BadRequestException;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import java.util.Set;

public final class SortValidator {

    private SortValidator() {}

    /**
     * Validates all requested sort fields against a strict per-entity allowlist,
     * applies domain defaults when unsorted, and appends primary-key tie-breaker ("id").
     */
    public static Pageable validate(Pageable pageable, Set<String> allowedFields, Sort defaultSort) {
        if (pageable == null || pageable.isUnpaged()) {
            return PageRequest.of(0, 20, ensureDeterministicIdSort(defaultSort));
        }

        Sort requestedSort = pageable.getSort();
        if (requestedSort.isUnsorted()) {
            return PageRequest.of(
                    pageable.getPageNumber(),
                    pageable.getPageSize(),
                    ensureDeterministicIdSort(defaultSort));
        }

        for (Sort.Order order : requestedSort) {
            String property = order.getProperty();
            if (!allowedFields.contains(property)) {
                throw new BadRequestException(
                        "Invalid sort field '" + property + "'. Allowed fields: " + allowedFields);
            }
        }

        return PageRequest.of(
                pageable.getPageNumber(),
                pageable.getPageSize(),
                ensureDeterministicIdSort(requestedSort));
    }

    private static Sort ensureDeterministicIdSort(Sort sort) {
        if (sort.getOrderFor("id") != null) {
            return sort;
        }
        Sort.Direction tieDirection = sort.stream()
                .findFirst()
                .map(Sort.Order::getDirection)
                .orElse(Sort.Direction.DESC);
        return sort.and(Sort.by(tieDirection, "id"));
    }
}
```

> **Spring Data 4.1 Import Alert:** In Spring Data Commons `4.1.0` (used by Spring Boot `4.1.0`), `PropertyReferenceException` lives in **`org.springframework.data.core`**`.PropertyReferenceException` — NOT the older `org.springframework.data.mapping` package!

src/main/java/ ... /exception/GlobalExceptionHandler.java (Sorting Exception Guard) MODIFIED

```java
import org.springframework.data.core.PropertyReferenceException;

// Safety net if any endpoint uses Spring Sort without SortValidator (returns 400 instead of 500)
@ExceptionHandler(PropertyReferenceException.class)
public ResponseEntity<ErrorResponse> handleInvalidSortProperty(
        PropertyReferenceException ex, HttpServletRequest request) {
    return buildResponse(HttpStatus.BAD_REQUEST,
            "Invalid sort property: " + ex.getPropertyName(), request);
}
```

4. Planned Code: Fixing Repositories & Wiring `SortValidator` in Services

Remove Rigid OrderBy Suffixes from Repositories (Dynamic Sort Compatibility) MODIFIED

```java
// BEFORE (Day 70): Rigid method name locks ORDER BY created_at DESC forever!
// List<Transaction> findBySourceAccountIdOrDestinationAccountIdOrderByCreatedAtDesc( ... );

// AFTER (Day 78): Clean derived method delegates sorting cleanly to Pageable.getSort()
// TransactionRepository.java
Page<Transaction> findBySourceAccountIdOrDestinationAccountId(
        Long sourceAccountId, Long destinationAccountId, Pageable pageable);

// LedgerRepository.java
Page<LedgerEntry> findByAccountId(Long accountId, Pageable pageable);

// AuditRepository.java
Page<AuditLog> findByEntityNameAndEntityId(String entityName, String entityId, Pageable pageable);
Page<AuditLog> findByPerformedBy_UserId(UUID userId, Pageable pageable);

// EmiRepository.java
Page<Emi> findByLoanId(Long loanId, Pageable pageable);
```

TransactionServiceImpl.java (Using SortValidator + Default Sort) MODIFIED

```java
private static final Set<String> ALLOWED_TX_SORT_FIELDS = Set.of(
        "id", "createdAt", "amount", "transactionType", "transactionStatus", "transactionReference"
);
private static final Sort DEFAULT_TX_SORT = Sort.by(Sort.Direction.DESC, "createdAt");

@Override
@Transactional(readOnly = true)
public PagedResponse<TransactionSummaryResponse> getTransactionHistory(Long accountId, Pageable pageable) {
    if (!accountRepository.existsById(accountId)) {
        throw new ResourceNotFoundException("Account not found with id: " + accountId);
    }

    Pageable validated = SortValidator.validate(pageable, ALLOWED_TX_SORT_FIELDS, DEFAULT_TX_SORT);

    Page<TransactionSummaryResponse> page = transactionRepository
            .findBySourceAccountIdOrDestinationAccountId(accountId, accountId, validated)
            .map(transactionMapper::toSummaryResponse);

    return PagedResponse.from(page);
}
```

5. Planned Code: Multi-Column Sort Support & Cache Key Safety

AccountServiceImpl.java (Validated Sort Inside @Cacheable Key) MODIFIED

```java
private static final Set<String> ALLOWED_ACCOUNT_SORT_FIELDS = Set.of(
        "id", "accountNumber", "balance", "accountType", "accountStatus", "branch", "createdAt"
);
private static final Sort DEFAULT_ACCOUNT_SORT = Sort.by(Sort.Direction.DESC, "createdAt");

// Notice: SortValidator runs inside service; invalid sort throws 400 BEFORE caching junk keys!
@Override
@Transactional(readOnly = true)
@Cacheable(value = "accountsByCustomer",
           key = "#customerId + ':' + #pageable.pageNumber + ':' + #pageable.pageSize + ':' +
#pageable.sort.toString()")
public PagedResponse<AccountSummaryResponse> getAccountsByCustomerId(Long customerId, Pageable pageable) {
    if (!customerRepository.existsById(customerId)) {
        throw new ResourceNotFoundException("Customer not found with id: " + customerId);
    }
    Pageable validated = SortValidator.validate(pageable, ALLOWED_ACCOUNT_SORT_FIELDS, DEFAULT_ACCOUNT_SORT);
    return PagedResponse.from(accountRepository.findByCustomerId(customerId, validated)
            .map(accountMapper::toSummaryResponse));
}
```

Multi-Column Sort Queries Out-of-the-Box: Spring Web automatically parses multiple `sort=` query parameters in priority order:
`GET /api/v1/transactions/account/1/history?page=0&size=10&sort=transactionStatus,asc&sort=amount,desc`
`SortValidator` validates *both* `transactionStatus` and `amount`, then produces SQL: `ORDER BY transaction_status ASC, amount DESC, id ASC`.

6. Verification Checklist (curl Tests)

Sort Test Case	HTTP Request	Expected Behavior
Single Column Descending	`GET ... /history?page=0&size=5&sort=amount,desc`	`200 OK`, highest amount first, tie-broken by `id`.
Multi-Column Sort	`GET ... /customers?sort=city,asc&sort=lastName,asc`	`200 OK`, grouped by city then alphabetical surname.
Invalid / Blocked Property	`GET ... /accounts/customer/1?sort=customer.user.password,asc`	`400 Bad Request` (rejected by `SortValidator`).
Default Sort Fallback	`GET /api/v1/emis/loan/1?page=0&size=12` (no `sort` param)	`200 OK`, sorted by `installmentNumber ASC`.